

Сравнение операторов “EXCEPT” и “NOT EXISTS” (с акцентом на проблему “NOT IN” и “NULL”)

Введение

При разработке SQL-запросов очень часто возникает задача поиска строк, отсутствующих в другом наборе данных. Для решения подобных задач используются различные подходы:

- ❑ **EXCEPT;** (или **MINUS** в Oracle)
- ❑ **NOT EXISTS;**
- ❑ **NOT IN.**
- ❑ **LEFT JOIN ... WHERE ... IS NULL**

> Особенно важной является проблема использования **NOT IN** при наличии значений **NULL**, поскольку это может привести к неожиданным результатам и логическим ошибкам.

Оператор EXCEPT

Оператор EXCEPT предназначен для получения строк, присутствующих в первом запросе и отсутствующих во втором.

Принцип работы:

- Выполняется первый запрос.
- Выполняется второй запрос.
- Из результата первого запроса исключаются строки второго.

Особенности EXCEPT:

- автоматически удаляет дубликаты;
- корректно работает с NULL;
- хорошо подходит для сравнения наборов данных;
- поддерживается PostgreSQL и SQL Server;
- отсутствует в MySQL.

- PostgreSQL поддерживает EXCEPT ALL;
- Microsoft SQL Server EXCEPT ALL не поддерживается.

Оператор NOT EXISTS

- NOT EXISTS проверяет отсутствие строк во вложенном подзапросе.
- Используется в коррелированных подзапросах.
- Очень мощный потому что он работает как: **ANTI JOIN**
 - операция, которая возвращает строки из левой таблицы, для которых нет совпадений в правой.

Особенности :

- | | |
|--|---|
| <ul style="list-style-type: none">• реализует anti join;• хорошо оптимизируется;• корректно работает с NULL;• эффективно использует индексы;• поддерживается всеми современными СУБД. | <ul style="list-style-type: none">• Анализ плана• PostgreSQL:• строит hash-таблицу;• ищет совпадения;• исключает найденные строки.• Данный алгоритм очень эффективен на больших таблицах. |
|--|---|

Недостатки NOT EXISTS

- Более сложный синтаксис по сравнению с NOT IN
- Коррелированные подзапросы труднее читать для новичков
- На старых версиях СУБД мог работать медленно (но современные оптимизаторы решают эту проблему)

Оператор NOT IN

- **NOT IN** проверяет отсутствие значения в наборе.
 - Главная проблема NOT IN связана с **NULL**.
 - Если во внутреннем запросе есть: NULL
 - результат может стать пустым.
 - **Производительность:** С подзапросами NOT IN заставляет PostgreSQL использовать неэффективные алгоритмы (например, последовательное сканирование). NOT EXISTS всегда генерирует более быстрый план выполнения.
- **Лучшая альтернатива**
 - Использование NOT EXISTS или LEFT JOIN ... WHERE ... IS NULL.

Сравнение

- Table: **Type_equipment**;
- Table: **Equipment**;

```
SELECT DISTINCT TE.Name  
FROM Type_equipment TE  
WHERE NOT EXISTS (  
    SELECT 1 FROM Equipment E  
    WHERE E.typeID = TE.typeID AND  
    E.InstallationDate >= CURRENT_DATE - INTERVAL  
    '5 years')  
ORDER BY TE.Name;
```

	total_type_equipment bigint
1	1000000

	total_equipment bigint
1	1000000

Сравнение

-> **Parallel Hash Anti Join** (cost=25203.75..43586.84 rows=416666)

```
SELECT DISTINCT TE.Name
FROM Type_equipment TE
WHERE NOT EXISTS (
    SELECT 1 FROM Equipment E
    WHERE E.typeID = TE.typeID AND
    E.InstallationDate >= CURRENT_DATE -
    INTERVAL '5 years')
ORDER BY TE.Name;
```

```
Output: te.name
Hash Cond: (te.typeid = e.typeid)
Buffers: shared hit=15782 read=9626
Worker 0:  actual time=193.301..398.923 rows=350613 loops=1
    Buffers: shared hit=5146 read=3394
Worker 1:  actual time=202.253..377.508 rows=297836 loops=1
    Buffers: shared hit=4950 read=2751
-> Parallel Seq Scan on public.type_equipment te (cost=0.00..13122.67 rows=416667 width=21) (actual time=0.238..62.423 rows=333333 loops=3)
    Output: te.typeid, te.name, te.registernumber, te.shortname
    Buffers: shared hit=311 read=8645
    Worker 0:  actual time=0.312..71.381 rows=350613 loops=1
        Buffers: shared hit=64 read=3076
    Worker 1:  actual time=0.236..56.503 rows=297836 loops=1
```

-> **Parallel Seq Scan on public.type_equipment te**

```
Output: e.typeid
Buckets: 524288  Batches: 1  Memory Usage: 15488kB
Buffers: shared hit=15413 read=981
Worker 0:  actual time=192.789..192.790 rows=94680 loops=1
    Buffers: shared hit=5053 read=318
Worker 1:  actual time=201.701..201.702 rows=88373 loops=1
```

-> **Parallel Seq Scan on public.equipment e**

```
Output: e.typeid
Filter: (e.installationdate >= (CURRENT_DATE - '5 years'::interval))
Rows Removed by Filter: 236814
Buffers: shared hit=15413 read=981
Worker 0:  actual time=0.049..164.547 rows=94680 loops=1
    Buffers: shared hit=5053 read=318
Worker 1:  actual time=0.040..174.353 rows=88373 loops=1
    Buffers: shared hit=4675 read=329
```

Planning:

Buffers: shared hit=209 read=12

Planning Time: 2.916 ms

Execution Time: 10302.175 ms

(62 rows)

Сравнение

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT TE.Name FROM Type_equipment TE
EXCEPT
SELECT TE2.Name FROM Type_equipment TE2
JOIN Equipment E ON E.typeID = TE2.typeID
WHERE E.InstallationDate >= CURRENT_DATE -
INTERVAL '5 years'
ORDER BY Name;
```

```
SetOp Except (cost=484284.42..490741.78 rows=1000000 width=17)
```

```
41752kB
5, temp read=10428 written=10448
rows=1291473 width=222) (actual time=0.035..3557.703 rows=1289557 loops=1)
read=9225
T* 1" (cost=0.00..28956.00 rows=1000000 width=21) (actual time=0.033..1246.096 rows=1000000 loops=1)
name, 0
read=8898
.type_equipment te (cost=0.00..18956.00 rows=1000000 width=17) (actual time=0.032..690.753 rows=1000000 loops=1)
hit=58 read=8898
T* 2" (cost=0.43..44096.76 rows=291473 width=21) (actual time=0.104..2097.915 rows=289557 loops=1)
name, 1
071 read=327
=0.43..41182.03 rows=291473 width=17) (actual time=0.103..2032.890 rows=289557 loops=1)
ue
hit=16071 read=327
Filter: (e.installationdate >= (CURRENT_DATE - '5 years'::interval))
Rows Removed by Filter: 710443
```

```
-> Nested Loop (cost=0.43..41182.03 rows=291473 width=17)
```

```
Cache Mode: logical
Hits: 289556 Misses: 1 Evictions: 0 Overflows: 0 Memory Usage: 1kB
```

```
-> Seq Scan on public.type_equipment te (cost=0.00..18956.00 rows=1000000 width=17)
```

```
Buffers: shared hit=4
```

```
Planning:
  Buffers: shared hit=205 read=16
Planning Time: 4.070 ms
Execution Time: 47708.150 ms
(42 rows)
```

Сравнение Plans

```
EXPLAIN ( ANALYZE, BUFFERS, VERBOSE)  
SELECT DISTINCT TE.Name  
FROM Type_equipment TE  
WHERE TE.typeID NOT IN (  
    SELECT E.typeID FROM equipment E  
    WHERE E.InstallationDate >=  
CURRENT_DATE - INTERVAL '5 years' )  
ORDER BY TE.Name;
```

Total rows: 1

Query complete 00:24:25.206

Planning:

Buffers: Shared?? hit=?? Read=??

Planning Time: ????? Hours

Execution Time: ??????.?? Hours

(??? rows)

Сравнение с индексами

```
CREATE INDEX idx_equipment_typeid_date ON Equipment (typeid, InstallationDate);
```

```
CREATE INDEX idx_type_equipment_name ON Type_equipment(Name);
```

```
SELECT DISTINCT TE.Name  
FROM Type_equipment TE  
WHERE NOT EXISTS (  
    SELECT 1 FROM Equipment E  
    WHERE E.typeid = TE.typeid AND  
    E.InstallationDate >= CURRENT_DATE - INTERVAL  
    '5 years')  
ORDER BY TE.Name;
```

```
EXPLAIN ( ANALYZE, BUFFERS, VERBOSE)  
SELECT TE.Name FROM Type_equipment TE  
EXCEPT  
SELECT TE2.Name FROM Type_equipment TE2  
JOIN Equipment E ON E.typeid = TE2.typeid  
WHERE E.InstallationDate >= CURRENT_DATE -  
INTERVAL '5 years'  
ORDER BY Name;
```

```

SELECT DISTINCT TE.Name
FROM Type_equipment TE
WHERE NOT EXISTS (
    SELECT 1 FROM Equipment E
    WHERE E.typeID = TE.typeID AND
    E.InstallationDate >= CURRENT_DATE -
    INTERVAL '5 years')
ORDER BY TE.Name;

```

-> Merge Anti Join (cost=0.85..54799.56 rows=999999 width=17)

-> Index Only Scan using idx_equipment_typeid_date on public.equipment e

-> Index Scan using pk_type_equipment on public.type_equipment te

```

Planning:
  Buffers: shared hit=267
Planning Time: 3.011 ms
Execution Time: 32192.015 ms
(24 rows)

```

```

Worker 0: actual time=193.301..398.923 rows=350613 loops=1
  Buffers: shared hit=5146 read=3304
Sort Method: external merge  Disk: 6176kB
Buffers: shared hit=4987 read=2751, temp read=772 written=773
Parallel Hash Anti Join (cost=25203.75..43586.84 rows=416666 width=17) (actual time=226.415..414.359 rows=333333 loops=3)
  Output: te.name
  Hash Cond: (te.typeid = e.typeid)
  Buffers: shared hit=15782 read=9626
  Worker 0:  actual time=193.301..398.923 rows=350613 loops=1
    Buffers: shared hit=5146 read=3304

```

```

-> Parallel Seq Scan on public.type_equipment te (cost=0.00..13122.67 rows=416667 width=21) (actual time=0.238..62.423 rows=333333 loops=3)
  Output: te.typeid, te.name, te.registernumber, te.shortname
  Buffers: shared hit=311 read=8645
  Worker 0:  actual time=0.312..71.381 rows=350613 loops=1
    Buffers: shared hit=64 read=3076
  Worker 1:  actual time=0.326..56.503 rows=297836 loops=1
    Buffers: shared hit=246 read=2422

```

```

buckets: 524288  batches: 1  memory usage: 15480kb
  Buffers: shared hit=15413 read=981
  Worker 0:  actual time=192.789..192.790 rows=94680 loops=1
    Buffers: shared hit=5053 read=318
  Worker 1:  actual time=201.701..201.702 rows=88373 loops=1

```

```

Filter: (e.installationdate >= (CURRENT_DATE - '5 years'::interval))
Rows Removed by Filter: 236814
  Buffers: shared hit=15413 read=981
  Worker 0:  actual time=0.049..164.547 rows=94680 loops=1
    Buffers: shared hit=5053 read=318
  Worker 1:  actual time=0.040..174.353 rows=88373 loops=1
    Buffers: shared hit=4675 read=329

```

```
EXPLAIN ( ANALYZE, BUFFERS, VERBOSE)
SELECT TE.Name FROM Type_equipment TE
EXCEPT
SELECT TE2.Name FROM Type_equipment TE2
JOIN Equipment E ON E.typeID = TE2.typeID
WHERE E.InstallationDate >= CURRENT_DATE -
INTERVAL '5 years'
ORDER BY Name;
```

```
SetOp Except (cost=461852.56..468291.60 rows=1000000 width=
```

```
rows=1000000 width=222) (actual time=43837.827..47396.589 rows=999999 loops=1) Output: "**SELECT* 1".name, (0)
mp read=10428 written=10448
ws=1291473 width=222) (actual time=43837.820..46252.787 rows=1289557 loops=1)
: 41752kB
25, temp read=10428 written=10448
rows=1291473 width=222) (actual time=0.035..3557.703 rows=1289557 loops=1)
ead=9225
CT* 1" (cost=0.00..28956.00 rows=1000000 width=21) (actual time=0.033..1246.096 rows=1000000 loops=1)
name, 0
8 read=8898
c.type_equipment te (cost=0.00..18956.00 rows=1000000 width=17) (actual time=0.032..690.753 rows=1000000 loops=1)
hit=58 read=8898
```

```
-> Merge Join (cost=2620.69..19980.43 rows=287809 width=17)
```

```
Output: te2.name
```

```
-> Index Only Scan using idx_equipment_typeid_date on public.equipment e
```

```
Rows Removed by Filter: 710443
Buffers: shared hit=16067 read=327
```

```
-> Index Scan using pk_type_equipment on public.type_equipment te2
```

```
hits: 209556 misses: 1 evictions: 0 overflows: 0 memory usage: 1kB
Buffers: shared hit=4
Output: te2.name, te2.typeid
Index Cond: (te2.typeid = e.typeid)
Buffers: shared hit=4
```

```
Planning Time: 0.375 ms
Execution Time: 16408.152 ms
(33 rows)
```

```
EXPLAIN ( ANALYZE, BUFFERS, VERBOSE)
SELECT DISTINCT TE.Name
FROM Type_equipment TE
WHERE TE.typeID NOT IN (
    SELECT E.typeID FROM equipment E
    WHERE E.InstallationDate >=
CURRENT_DATE - INTERVAL '5 years' )
ORDER BY TE.Name;
```

Planning:

Buffers: Shared?? hit=?? Read=??

Planning Time: ????? Hours

Execution Time: ??????.?? Hours

(??? rows)

Выводы

Запрос	Время без индексов	Время с индексами
EXCEPT	47.708 сек	16.408 сек
NOT EXISTS	10.302 сек	32.192 сек
NOT IN	???.?? сек	???.?? сек

Характеристика	NOT EXISTS	EXCEPT
Поддержка MySQL	Да	Нет
Работа с NULL	Отличная	Отличная
Anti Join	Да	Косвенно
Дубликаты	Сохраняет	Удаляет
Читаемость	Средняя	Очень высокая
Производительность	Очень высокая	Высокая

СПАСИБО!

ЕСТЬ ВОПРОСЫ?